

**Cloud Flow: A Multi-Pod, Multi-Container Image Processing & Inference Pipeline on
Kubernetes**

Hamed Diakite

Department of Computer Science, West Chester University of Pennsylvania

CSC 581-01: Cloud Engineering

Dr. Ngo, Graduate Coordinator & Course Lecturer

September 27, 2026

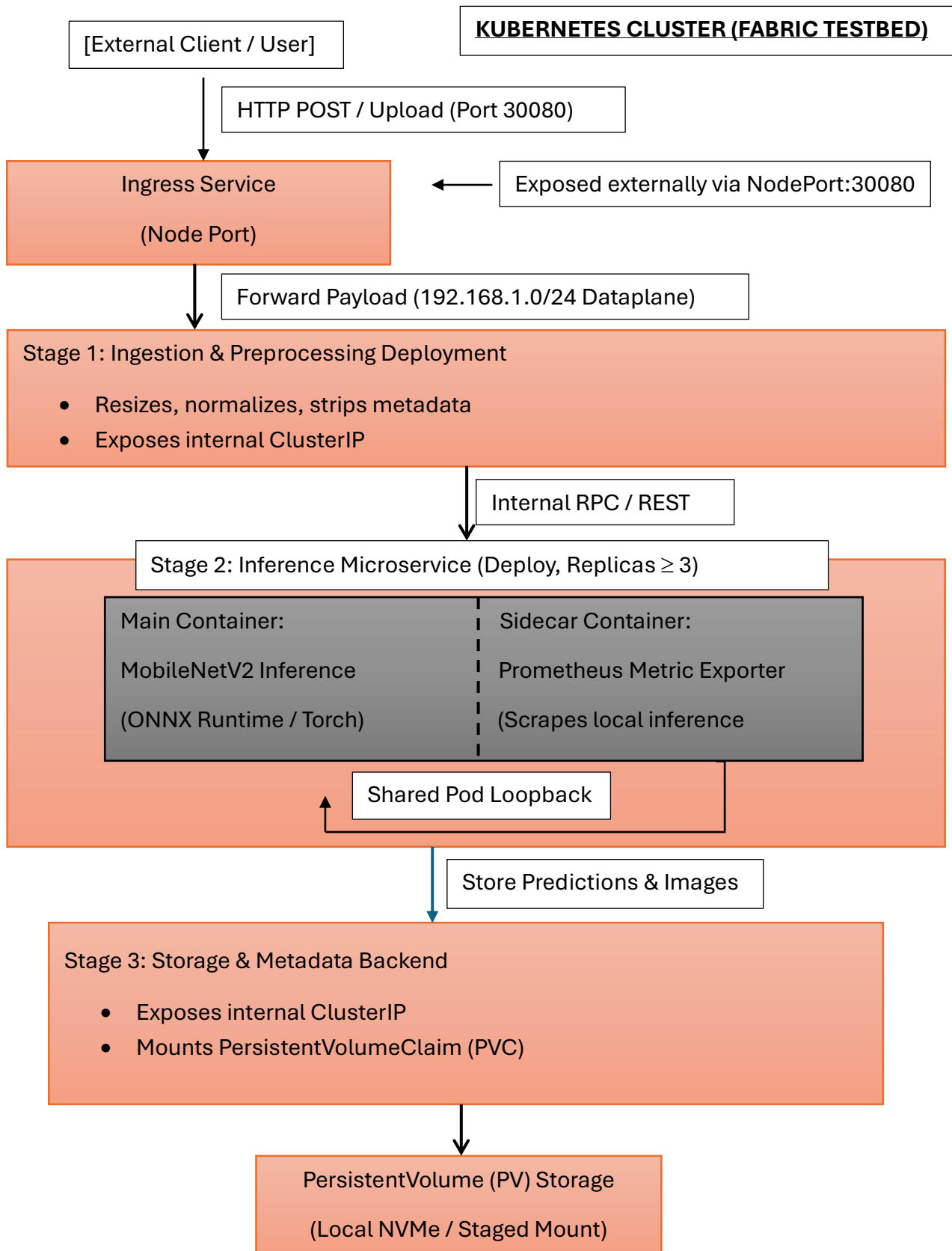
Chapter 1: Project Vision & High-Level Architecture

1.1 Project Vision

Modern computer vision workloads require decoupling compute-intensive model inference from data ingestion, normalization, and persistent indexing. Monolithic machine learning architectures struggle to balance varying resource demands: image transformations consume dynamic CPU and memory bursts, neural inference requires predictable, scaled compute resources, and persistence layers demand strict data durability.

The primary objective of **CloudFlow** is to build a distributed, microservice-oriented image processing and inference pipeline hosted on an RKE2 Kubernetes cluster deployed over the FABRIC testbed. The system ingests raw image data, preprocesses and standardizes incoming payloads, executes deep-learning classification via pre-trained vision models (e.g., MobileNetV2), and writes prediction metadata to durable storage. By decomposing these stages into isolated Kubernetes Deployments, each tier scales independently according to real-time load while remaining fault tolerant.

1.2 10,000-Foot View Architecture



1.3 Architecture Elaboration

The CloudFlow architecture segregates responsibilities across network boundaries and workload boundaries to enforce strict isolation:

- **Boundary Isolation & Ingress:** The external network touches only the Ingress Preprocessing Service via a single `NodePort` entry point (e.g., port 30080). Internal stages communicate exclusively through Kubernetes `ClusterIP` services, shielding inference workers and storage units from untrusted outside traffic.
- **Decoupled Dataflow:** The preprocessing worker sanitizes images, rescales resolutions to uniform tensors (224×224), and routes payloads to an internal cluster endpoint.
- **Multi-Container Co-location:** The inference stage combines a core prediction engine with an autonomous monitoring sidecar in the same Pod. Because containers within a Pod share identical networking namespaces and loopback interfaces (`localhost`), the sidecar gathers performance telemetry (latency, memory overhead, classification accuracy) without intercepting ingress traffic or requiring changes to the model codebase.
- **State Decoupling:** Inference pods remain stateless, allowing horizontal scaling (≥ 3 replicas) and dynamic self-healing across worker nodes. State persistence is relegated to a dedicated storage service mapped to a `PersistentVolumeClaim` (PVC).

Chapter 2: Proposed Implementation Plan

2.1 Pod and Container Decomposition

The pipeline consists of three discrete Kubernetes deployments:

1. Ingress & Preprocessing Pod (cloudflow-preprocessor):

- *Base Image*: Python 3.11-slim, Pillow, OpenCV.
- *Role*: Validates inbound MIME types, compresses image payloads, rescales arrays to normalized dimensions, and dispatches data to inference workers.
- *Scaling Target*: 2 replicas.

2. Inference Pod (cloudflow-inference — Multi-Container Pod):

- *Main Container (classifier)*: Python runtime running ONNX Runtime / PyTorch serving a lightweight, pre-trained MobileNetV2 classifier. Scaled to ≥ 3 replicas to meet project scaling targets.
- *Sidecar Container (metrics-collector)*: Lightweight container running a Python/Go Prometheus exporter tailing local execution logs or polling localhost:8080/metrics to expose inference latency and compute overhead.

3. Storage & Metadata Pod (cloudflow-storage):

- *Base Image*: SQLite / Lightweight REST store.

- *Role:* Receives classified label records and bounding metadata, appending them to disk alongside raw image references.

2.2 Kubernetes Manifests & Resource Specification

- **Deployments:** Three YAML manifests located in `deployments/` specifying Pod replica counts, resource limits (e.g., `cpu: "500m"`, `memory: "512Mi"`), container port mappings, and container lifecycle probes (`readinessProbe` and `livenessProbe`).
- **Services:**
 - `cloudflow-ingress-svc`: Type `NodePort` mapping node port 30080 to container port 8080.
 - `cloudflow-inference-svc`: Type `ClusterIP` exposing port 5000 internally for round-robin balancing across the ≥ 3 inference pods.
 - `cloudflow-storage-svc`: Type `ClusterIP` exposing port 6379 / 8000 strictly to cluster services.
- **Storage Primitives:** A `PersistentVolumeClaim` (`pvc.yaml`) bound to a local host-path or dynamic FABRIC storage provisioner (`pv.yaml`) mapped to `/data/cloudflow` to ensure data survivability across pod restarts.

2.3 FABRIC Infrastructure & Cluster Topology

The deployment runs on a 3-node RKE2 Kubernetes cluster provisioned via the course's FABlib infrastructure scripts:

- **Node 1 (node1):** IP 192.168.1.1 — Control plane (rke2-server), running cluster management and hosting ingress endpoints.
- **Node 2 (node2):** IP 192.168.1.2 — Worker node (rke2-agent), running preprocessing and inference workloads.
- **Node 3 (node3):** IP 192.168.1.3 — Worker node (rke2-agent), running scaled inference replicas and persistent backend pods.
- **Dataplane Networking:** The nodes communicate across a dedicated FABRIC Layer 2 network (192.168.1.0/24). Cluster ingress is validated using local SSH port forwarding (-L 5555:127.0.0.1:30080) through the Bastion jump host.

2.4 Verification & Testing Plan

- **End-to-End Data Pipeline:** Ingest test images using curl against http://127.0.0.1:30080/upload and confirm classification labels return in JSON.
- **Self-Healing & Scalability:** Execute `kubectl scale deployment cloudflow-inference -- replicas=3`, verify distributed pod placement across nodes, and delete one active pod (`kubectl delete pod <pod-name>`) to demonstrate automated ReplicaSet recovery.
- **Sidecar Telemetry:** Execute `curl http://localhost:9090/metrics` inside the multi-container pod to verify real-time metrics gathering over loopback.
- **Storage Survivability:** Delete the storage pod and redeploy it to verify that persisted images and classification logs remain intact on the mounted volume.